

BubbleSort – Fehleranalyse (IHK-Stil)

Typische Fehlerbilder aus Prüfungen (2022–2024): Off-by-One in Schleifen, falscher Vergleich (auf/absteigend), fehlender Tausch, falscher Index $j+1$, nicht zurückgesetztes Flag bei der optimierten Version u.v.m.

F1 – Off-by-One: Innere Schleife läuft zu weit

Aufgabe: Finden und korrigieren Sie den Fehler. Begründen Sie kurz.

Fehlerhafter Pseudocode

```
Funktion bubbleSort(a: Array) : void
  für i = 0 bis länge(a) - 1
    für j = 0 bis länge(a) - 1 - i
      wenn a[j] > a[j + 1] dann
        t = a[j]; a[j] = a[j+1]; a[j+1] = t
      ende wenn
    ende für
  ende für
Ende Funktion
```

Korrektur (IHK-Pseudocode)

```
Funktion bubbleSort(a: Array) : void
  für i = 0 bis länge(a) - 2
    für j = 0 bis länge(a) - 2 - i
      wenn a[j] > a[j + 1] dann
        t = a[j]; a[j] = a[j+1]; a[j+1] = t
      ende wenn
    ende für
  ende für
Ende Funktion
```

Begründung

Die Referenz `a[j+1]` darf nie über das Feldende laufen. Die letzten Vergleiche jeder Runde entfallen, weil das größte Element bereits „fixiert“ ist. Daher `n-2` statt `n-1`.

Java-ähnlich

```
void bubbleSort(int[] a){
    for(int i=0;i < a.length-1;i++){
        int t=a[i]; a[i]=a[i+1]; a[i+1]=t;
    }
}
```

F2 – Falscher Vergleich (absteigend statt aufsteigend – oder umgekehrt)

Aufgabe: Der Algorithmus sortiert in die falsche Richtung. Passen Sie den Vergleich an.

Fehlerhafter Pseudocode

```
Funktion bubbleSortAufsteigend(a: Array) : void
  für i = 0 bis länge(a) - 2
    für j = 0 bis länge(a) - 2 - i
      wenn a[j] < a[j + 1] dann    // falsches Zeichen für aufsteigend
        t = a[j]; a[j] = a[j+1]; a[j+1] = t
      ende wenn
    ende für
  ende für
Ende Funktion
```

Korrektur

```
... wenn a[j] > a[j + 1] dann ...    // aufsteigend richtig
```

Begründung

Für **aufsteigend** müssen größere Werte nach rechts wandern → tauschen bei $a[j] > a[j+1]$.
Für **absteigend** entsprechend $a[j] < a[j+1]$.

F3 – Tausch vergessen / falsche Tauschreihenfolge

Aufgabe: Der Code vergleicht richtig, aber die Werte werden nicht (korrekt) getauscht.

Fehlerhafter Pseudocode

```
wenn a[j] > a[j+1] dann
    a[j] = a[j+1]
    a[j+1] = a[j]    // verliert den ursprünglichen Wert
ende wenn
```

Korrektur

```
wenn a[j] > a[j+1] dann
    t = a[j]
    a[j] = a[j+1]
    a[j+1] = t
ende wenn
```

Begründung

Ohne temporäre Variable geht der erste Wert verloren. Immer $t = a[j]$ zwischenspeichern.

F4 – Optimierter BubbleSort: Flag nicht korrekt gesetzt

Aufgabe: Frühzeitiger Abbruch funktioniert nicht. Reparieren Sie das Flag-Handling.

Fehlerhafter Pseudocode

```
getauscht = true
solange getauscht
    // getauscht wird hier NICHT auf false zurückgesetzt
    für i = 0 bis länge(a) - 2
        wenn a[i] > a[i+1] dann
            t = a[i]; a[i] = a[i+1]; a[i+1] = t
            getauscht = true
        ende wenn
    ende für
ende solange
```

Korrektur

```
getauscht = true
solange getauscht
    getauscht = false // ← wichtig: zu Beginn der Runde zurücksetz
    für i = 0 bis länge(a) - 2
        wenn a[i] > a[i+1] dann
            t = a[i]; a[i] = a[i+1]; a[i+1] = t
            getauscht = true // es gab mindestens einen Tausch
        ende wenn
    ende für
ende solange
```

Begründung

Wenn zu Beginn einer Runde nicht auf `false` zurückgesetzt wird, bleibt das Flag ggf. dauerhaft `true` → Endlosschleife oder nutzloser Mehraufwand.

F5 – Startindex der inneren Schleife falsch gewählt

Aufgabe: Der erste Vergleich jeder Runde wird ausgelassen.

Fehlerhafter Pseudocode

```
für j = 1 bis länge(a) - 2 - i    // startet bei 1 statt 0
    wenn a[j] > a[j+1] dann ...
ende für
```

Korrektur

```
für j = 0 bis länge(a) - 2 - i
```

Begründung

BubbleSort vergleicht stets j mit j+1 ab dem ersten Paar (Index 0 und 1).

F6 – Länge/Index verwechseln (z. B. length vs. länge())

Aufgabe: Syntax/Bezeichner sind inkonsistent – korrigieren Sie die Längen-Abfrage.

Fehlerhafter Pseudocode

```
für i = 0 bis a.length() - 2    // Methodenaufruf statt Eigenschaft
...
ende für
```

Korrektur (IHK-Pseudocode)

```
für i = 0 bis länge(a) - 2
```

Java-Hinweis

In Java ist es `a.length` (Feld-Eigenschaft), nicht `a.length()` (Methode). Im IHK-Pseudocode nutzen wir einheitlich `länge(a)`.

F7 – String-Vergleich: lexikographisch statt „==“

Aufgabe: Strings werden mit == verglichen. Korrigieren Sie den lexikographischen Vergleich.

Fehlerhafter Pseudocode

```
wenn namen[j] == namen[j+1] dann ... // falsches Kriterium zum Sortieren
```

Korrektur

```
wenn namen[j] > namen[j+1] dann    // lexikographisch (IHK-Pseudocode)
    tauschen ...
ende wenn
```

Java-ähnlich

```
if (namen[j].compareTo(namen[j+1]) > 0) { ... }
```

F8 – Indizes vertauscht (i/j falsch verwendet)

Aufgabe: Die falsche Schleifenvariable wird im Vergleich genutzt.

Fehlerhafter Pseudocode

```
für i = 0 bis länge(a) - 2
  für j = 0 bis länge(a) - 2 - i
    wenn a[i] > a[i+1] dann // sollte j sein
      ...
    ende wenn
  ende für
ende für
```

Korrektur

```
... wenn a[j] > a[j+1] dann ...
```

Begründung

Verglichen werden stets j und j+1 (Nachbarpaare). i steuert nur die Rundenlänge.

F9 – Optimierter BubbleSort: falsche Schleifenform

Aufgabe: Das Flag wird korrekt gesetzt, aber eine for-Schleife verhindert den Abbruch.

Fehlerhafter Pseudocode

```
getauscht = true
für runde = 1 bis länge(a) - 1
    getauscht = false
    für i = 0 bis länge(a) - 2
        ...
        wenn ... dann getauscht = true
    ende für
    // hier findet KEIN Abbruch statt, obwohl getauscht = false sein kann
ende für
```

Korrektur

```
getauscht = true
solange getauscht
    getauscht = false
    für i = 0 bis länge(a) - 2
        ...
        wenn ... dann getauscht = true
    ende für
ende solange
```

Begründung

Nur die **bedingte** Schleife (solange/while) kann frühzeitig beenden, wenn kein Tausch erfolgt.

F10 – Randfall: Leeres Feld oder Länge 1

Aufgabe: Code bricht bei leeren Feldern, prüfen/absichern.

Absicherung (optional, sauberer Stil)

```
Funktion bubbleSort(a: Array) : void
    wenn länge(a) <= 1 dann
        gib nichts zurück    // bereits „sortiert“
    ende wenn
    // ... Standard-Algorithmus ...
Ende Funktion
```

Mini-Check (wie in der Prüfung)

- Warum endet die innere Schleife bei $n - 2 - i$?
- Was passiert, wenn Sie j fälschlich bei 1 starten?
- Wie lautet der korrekte Vergleich für absteigend?
- Weshalb braucht man eine temporäre Variable beim Tauschen?
- Wie verhindert die Abbruchbedingung unnötige Runden?

Merksätze: (1) Nach jeder Runde steht das größte Element des unsortierten Bereichs am Ende. (2) Innere Schleife: bis $n-2-i$. (3) Aufsteigend tauschen bei $>$, absteigend bei $<$. (4) Optimiert: getauscht pro Runde auf `false` setzen – nur bei Tausch wieder `true`.